8

אוטומציות והוקים: Claude עובד גם כשלא מבקשים

עד עכשיו כל עבודה התחילה בבקשה שלכם. עכשיו מלמדים את Claude להתחיל לבד:
לפי אירוע, לפי שעון, לפי מטרה, או בלחיצה על קישור.

מה חסר עדיין



עד עכשיו

אתם מבקשים, Claude עושה. כל עבודה מתחילה ממכם: מקלידים, מחכים, בודקים.

⚡

מה שחסר

עבודה שמתחילה לבד. שומר שמונע טעויות בלי שתזכרו. בדיקה כל חצי שעה. סוכן שממשיך עד שהמשימה באמת גמורה.

נועה

היא לא רוצה לזכור לבדוק שהסוכן לא שינה את סיכומי ההרצאות המקוריים. והיא לא רוצה לשבת מול המסך כל זמן שהוא עובד. אוטומציה פותרת את שני הדברים.

מי מתחיל את העבודה?

הוק

קורה אירוע (Claude עומד לערוך קובץ), ופקודה רצה אוטומטית.

לולאה

עובר זמן (כל 10 דקות), ובקשה רצה שוב.

מטרה

התנאי עוד לא מתקיים, אז Claude ממשיך לעבוד.

ללא ממשק

סקריפט שלכם מריץ Claude בשורת פקודה, בלי שיחה.

ערוץ

הודעה מבחוץ (טלגרם, CI) נכנסת לשיחה שפתוחה.

קישור

לחיצה על כתובת פותחת Claude עם בקשה מוכנה.

הוק: פעולה אוטומטית ברגע מסוים

🚪

דימוי

דלת אוטומטית בכניסה לחנות. אף אחד לא מחליט לפתוח אותה בכל פעם: יש חיישן, ויש כלל. הוק הוא החיישן והכלל של Claude.

🪝

הגדרה

פקודה שאתם כותבים פעם אחת, ו-Claude מריץ אותה בכל פעם שמתרחש האירוע. בלי להסתמך על כך שהמודל "יזכור".

למה זה חשוב לכם

הוראה ב-CLAUDE.md היא בקשה יפה. הוק הוא כלל שנאכף. אם חשוב שקובץ לעולם לא ישתנה, לא מבקשים: חוסמים.

שלושה חלקים: מתי, על מה, מה לעשות

PreToolUse
מתי: האירוע. כאן, רגע לפני שימוש בכלי

matcher
על מה: על אילו כלים זה חל. כאן עריכה או כתיבת קובץ

command
מה: הפקודה שרצה

```
settings.json
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          { "type": "command",
            "command": "..." }
        ]
      }
    ]
  }
}
```

⚡ ההגדרה נשמרת בקובץ ההגדרות: `.claude/settings.json` לפרויקט הזה, או `~/.claude/settings.json` לכל הפרויקטים.

האירועים שכדאי להכיר

אירוע	מתי הוא קורה	לדוגמה
PreToolUse	לפני שימוש בכלי. אפשר לחסום	לא לערוך קבצי מקור
PostToolUse	אחרי שכלי הצליח	לעצב קובץ שנערך
Notification	Claude מחכה לכם	התראה על המסך
Stop	Claude סיים לענות	בדיקה לפני עצירה
SessionStart	תחילת שיחה	להזכיר כללים

⚡ יש עוד אירועים רבים (בתיעוד של hooks). אלה חמשת הראשונים לתחילת הדרך.

התראה כש-Claude מחכה לכם

1. מדביקים בחלונית של Claude Code את הבקשה למטה
2. Claude כותב את ההוק לקובץ ההגדרות, ועוצר. **קוראים** מה נכתב
3. מקלידים `/hooks` . ההוק מופיע ברשימה תחת Notification

הוסף לקובץ ההגדרות האישי שלי (~/.claude/settings.json) הוק על האירוע Notification, שמציג התראה על המסך בנוסח "Claude Code needs your attention". המחשב שלי: Mac. אל תמחק הגדרות קיימות, ועצור כדי שאראה מה כתבת.

⚡ ב-Windows או ב-Linux? פשוט כתבו בבקשה את מערכת ההפעלה שלכם, ו-Claude יתאים את הפקודה.

מה נכתב, ואיך בודקים

1. מבקשים מ-Claude משהו שדורש אישור (למשל ליצור קובץ)
2. עוברים לחלון אחר ומחכים כמה שניות
3. מופיעה התראה על המסך

```
~/.claude/settings.json
{ "hooks": { "Notification": [ {
    "matcher": "",
    "hooks": [ {
      "type": "command",
      "command": "osascript -e '...'"
    } ]
} ] } }
```

✅ **סיימתם כש:** קיבלתם התראה כש-Claude חיכה לכם.

איך הוק עוצר את Claude: שלושה קודי יציאה

✅

0

הכול בסדר, ממשיכים. אין התנגדות.

⛔

2

חוסמים. הטקסט שהוק כתב ל-stderr מגיע ל-Claude כהסבר, והוא מתאים את הדרך.

⚠️

כל קוד אחר

שגיאה לא חוסמת. הפעולה נמשכת, ומוצגת הודעת שגיאה.

הדימוי

מאבטח בכניסה. מנהל אומר "עבור" (0) או "עצור, ולא מכניסים בגלל..." (2). Claude שומע את ההסבר ומחפש דרך אחרת.

שומר: אסור לגעת בקבצי המקור

מה הוא עושה
קורא את שם הקובץ ש-Claude עומד לערוך

אם הקובץ בתוך notes/
כותב הסבר וחוזר עם קוד 2: חסימה

אחרת
קוד 0: ממשיכים כרגיל

.claude/hooks/protect-notes.sh

```
#!/bin/bash
INPUT=$(cat)
FILE=$(echo "$INPUT" | jq -r \
  '.tool_input.file_path // empty')

if [[ "$FILE" == */notes/* ]]; then
  echo "Blocked: notes/ is read-only" >&2
  exit 2
fi
exit 0
```

⚡ הסקריפט קורא את הקלט עם jq. אם חסר לכם: `brew install jq`. אפשר לבקש מ-Claude להתקין ולהסביר.

יוצרים ובודקים

1. מדביקים את הבקשה למטה. Claude כותב סקריפט והגדרה, ועוצר
2. קוראים את שניהם. מאשרים להפעיל את הסקריפט (`chmod +x`)
3. בודקים: "הוסף שורה לקובץ notes/syllabus-dsp.md"

צור הוק PreToolUse על Edit ו-Write בהגדרות הפרויקט (.claude/settings.json). הוא מריץ סקריפט claude/hooks/protect-notes.sh. שחוסם (קוד יציאה 2) כל עריכה של קובץ שנמצא בתיקייה notes/, עם הודעה "Blocked: notes/ is read-only". הסקריפט קורא את הנתיב עם jq. הפוך אותו להרצה ועצור כדי שאראה.

✅ **סיימתם כש:** הבקשה לערוך קובץ ב-notes/ נחסמה, ו-Claude הסביר לכם למה.

מה עוד אפשר, ומה שחייבים לדעת

עוד סוגי הוקים

- **פקודה**: סקריפט (מה שבנינו)
- **פרומפט**: מודל קטן שופט בשאלה, למשל "האם המשימה גמורה?"
- **כתובת**: שולח את האירוע לשירות באינטרנט

בטיחות

- הוק רץ **עם ההרשאות שלכם**. קוראים לפני שמוסיפים
- לא מדביקים הוק מאתר לא מוכר
- הוק בהגדרות הפרויקט רץ גם אצל מי שפותח אותו, בדקו מה בתיקיות שמורידים

⚡ הוק שמבוסס על פרומפט הוא הבסיס של `/goal`, שנראה בהמשך.

לחזור על בקשה לפי שעון

עם מרווח

`/loop 5m` ובקשה. רצה כל 5 דקות.

בלי מרווח

Claude בוחר לבד כמה לחכות: קצר כשיש פעילות, ארוך כשהכול שקט.

בלי כלום

`/loop` לבד מריץ בדיקת תחזוקה מובנית.

```
/loop 10m check if new files appeared in notes/ and tell me what they are
```

⚡ נסו: הריצו, ואז גררו קובץ חדש ל-notes/. בפעם הבאה ש-Claude יבדוק, הוא ידווח.

תזכורת חד-פעמית, ואיך מנהלים

תזכורת חד-פעמית

תזכיר לי ב-15:00 לשלוח את הדוח למנחה

רצה פעם אחת ונמחקת.

ניהול

- **רשימה**: "אילו משימות מתוזמנות יש לי?"
- **ביטול**: "בטל את בדיקת הקבצים"
- **תפוגה**: לולאה חוזרת נעצרת אחרי 7 ימים

⚡ מעדיפים לא להסתמך על חלון פתוח? בהמשך: משימות בענן ובאפליקציית שולחן העבודה.

שלוש דרכים לתזמן, ואיפה כל אחת רצה

	/loop	שולחן עבודה	ענן
רץ על	המחשב שלכם	המחשב שלכם	הענן
דורש חלון פתוח	כן	לא	לא
דורש מחשב דלוק	כן	כן	לא
גישה לקבצים שלכם	כן	כן	לא
מרווח מינימלי	דקה	דקה	שעה

⚡ בדיקה מהירה בזמן עבודה: /loop. משהו קבוע שצריך קבצים מקומיים: שולחן עבודה. משהו שחייב לרוץ גם כשהמחשב כבוי: ענן.

להמשיך עד שהמשימה באמת גמורה

1. כותבים `/goal` ותנאי שאפשר לבדוק
2. Claude מתחיל לעבוד מיד
3. אחרי כל תור, מודל קטן שופט: התנאי התקיים?
4. לא התקיים? Claude ממשיך. כן? המטרה נסגרת

```
/goal summary.html exists and has a section for every topic in notes/syllabus-dsp.md, or stop after 8 turns
```

תנאי טוב: אפשר להוכיח אותו

תנאי עמום

"הסיכום טוב"
איך השופט יודע? אין מה לבדוק.

תנאי חד

"summary.html קיים, יש בו סעיף לכל אחד מעשרת הנושאים, ושום קובץ ב-/notes לא השתנה. או עצור אחרי 8 תורות"
אפשר לראות ולבדוק.

⚡ תנאי טוב כולל: **מצב סופי אחד** (קובץ שקיים, בדיקה שעוברת) • **איך מוכיחים** ("git status נקי") • **מה אסור לשנות** ("בלי לערוך קבצי מקור")

לבדוק, לבטל, ולבחור בין הכלים

פקודות

- `/goal` לבד: מצב נוכחי וסיבת השופט
- `/goal clear` : מבטל
- סיום השיחה (`/clear`) מנקה גם אותו

מתי מה

- **/goal**: ממשיך עד שתנאי מתקיים
- **/loop**: חוזר לפי שעון
- **Stop hook**: הכלל שלכם, בכל שיחה

⚡ /goal הוא קיצור: הוק פרומפט שחי רק בשיחה הנוכחית. הוק Stop בקובץ ההגדרות חי בכל שיחה.

Claude משורת פקודה: claude -p

```
claude -p "What does the syllabus say about the final exam?"
```

💬 בלי שיחה

שואלים, מקבלים תשובה, ושום דבר לא נשאר פתוח.

📜 בתוך סקריפט

אפשר לשים אותה בקובץ, ולהריץ מתי שרוצים.

🚦 קוד יציאה

0 כשהצליח, אחרת שגיאה. סקריפטים יכולים להגיב.

⚡ הדגמה בלבד. פותחים טרמינל בתוך VS Code (Terminal → New Terminal) ומריצים בתיקייה /agents-lab.

לשלב קבצים, כלים ופלט מסודר

```terminal
cat notes/syllabus-dsp.md | claude -p "list every exam as a table"
claude -p "Summarize notes/project-eeg.md" --allowedTools "Read"
claude -p "List the topics" --output-format json
```

צינור (pipe)
מעבירים תוכן של קובץ לתוך Claude

--allowedTools
אילו כלים מותרים בלי לשאול. כאן רק קריאה

--output-format json
פלט מובנה לתוכנית אחרת

הודעות מבחוץ שנכנסות לשיחה פתוחה



מה זה

הודעה מבחוץ נכנסת לשיחה שכבר פתוחה: טלגרם, כשל בבדיקות, התראה. Claude מגיב ויכול לענות באותו ערוץ.



דימוי

פקס של פעם, שמגיע ישר לשולחן העבודה שלכם.

לאן

טלגרם, דיסקורד, iMessage (ניסוי)

צריך

חשבון claude.ai ושיחה פתוחה

⚡ בחשבונות צוות וארגון, המנהל צריך להפעיל את זה.

הדגמה בטוחה: fakechat, ואיך נעולים

```terminal
/plugin install fakechat@claude-plugins-official

claude --channels plugin:fakechat@claude-plugins-official

# then open http://localhost:8787 in your browser
```

🧪

הדגמה

חלון צ׳אט מקומי: מקלידים בדפדפן, ההודעה מגיעה ל-Claude והתשובה חוזרת. אין חשבון חיצוני.

🔒

אבטחה

רק מי שברשימת ההיתרים (allowlist) יכול לשלוח הודעות. אחרים מסוננים בשקט.

קישור שפותח Claude עם בקשה מוכנה

```
claude-cli://
claude-cli://open
  ?cwd=/Users/noa/agents-lab
  &q=Review%20notes%2Fsyllabus-dsp.md
```

📁

cwd

באיזו תיקייה לפתוח. נתיב מלא.

✍️

q

הבקשה שתופיע בתיבה. מקודדת ככתובת.

🔒

הכלל

הבקשה **לא נשלחת** עד שאתם לוחצים Enter.

ניסוי של 30 שניות

1. מחליפים את הנתיב בנתיב המלא של `agents-lab/` אצלכם
2. מריצים בטרמינל (Mac) את הפקודה למטה
3. נפתח חלון חדש עם Claude Code והבקשה מוכנה

```
open "claude-cli://open?cwd=/Users/YOURNAME/Desktop/agents-lab&q=List%20the%20files%20in%20notes%2F"
```

⚡ ב-Windows: `Start-Process "claude-cli://open?..."` . ב-GitHub קישור כזה לא נלחץ בתוך README, לכן שמים אותו בבלוק קוד.

איזה כלי בוחרים

אני רוצה ש...	הכלי
Claude לא ייגע בקבצים מסוימים	הוק PreToolUse
אקבל התראה כשהוא מחכה לי	הוק Notification
הוא יבדוק משהו כל כמה דקות	/loop
הוא ימשיך עד שהמשימה גמורה	/goal
סקריפט שלי יריץ אותו	claude -p
הודעה מבחוץ תיכנס לשיחה	ערוץ
לחיצה תפתח אותו עם בקשה	קישור עמוק

Workshop 8

15:00

בונים אוטומציה אחת משלכם

agents-lab/ 🗂️ | 🪝 הוק | ⏰ לולאה | 🎯 מטרה | ✅ בדיקה

בוחרים מסלול אחד ובונים אותו על הסוכנים מהמודול הקודם.

🪝 מסלול א׳: שומר

הוק שחוסם עריכה ב-notes/, והוק התראה.

⏰ מסלול ב׳: שומר זמן

לולאה שמדווחת על קבצים חדשים, ותזכורת.

🎯 מסלול ג׳: עד הסוף

מטרה שמריצה את החוקר והבונה עד ש-summary.html שלם.

חמישה צעדים

1. בוחרים מסלול אחד, ופותחים את `agents-lab/`
2. מדביקים את הבקשה של המסלול (בשקף הבא) ומחכים ש-Claude יעצור
3. **קוראים** מה נכתב: איזה אירוע, איזו פקודה, מה היא עושה
4. בודקים עם בקשה שאמורה להפעיל אותה, ובקשה שלא
5. שואלים: "מה יכול להשתבש?", ומתקנים תנאי אחד

הבקשות לשלושת המסלולים

מסלול א׳: צור הוק PreToolUse על Edit ו-Write שחוסם (קוד 2) עריכת קבצים בתיקייה notes/ עם הודעה ברורה. כתוב סקריפט ב-claude/hooks/. ועצור כדי שאראה.

מסלול ב׳: loop/ 10m בדוק אם נוספו קבצים חדשים לתיקייה notes/. אם כן, ציין את שמם בשורה אחת. אם לא, כתוב ״אין חדש״.

מסלול ג׳: goal/ summary.html קיים וכולל סעיף לכל נושא שהחוקר מצא ב-notes/, בלי לערוך אף קובץ ב-notes/, או עצור אחרי 8 תורות.

חומר אחר? מחליפים את שם התיקייה והקובץ. הרעיון זהה.

איך יודעים שזה עובד

הוק
הבקשה לערוך קובץ ב-notes/ נחסמה, וההסבר הגיע ל-Claude

לולאה
קובץ חדש ב-notes/ הופיע בדיווח, גם בלי שביקשתם

מטרה
בסוף יש תוצר שלם, ואפשר לראות את סיבת השופט עם /goal

✅ **סיימתם כש:** אוטומציה אחת רצה אצלכם בפועל, ואתם יכולים להסביר בשלוש מילים איזה אירוע מפעיל אותה.

⚡ נתקעתם? שאלו את Claude: "למה האוטומציה לא קרתה, ומה לשנות?"

טעויות נפוצות

- **הוק בלי לקרוא.** הוא רץ עם ההרשאות שלכם
- **חסימה רחבה מדי.** matcher ריק חוסם הכול
- **תנאי עמום ב-goal/.** השופט לא יודע מתי לעצור
- **בלי גבול.** תמיד "או עצור אחרי N תורות"
- **לולאה בלי חלון.** היא רצה רק כשהסשן פתוח
- **קישור או ערוץ ללא בדיקה.** קוראים לפני שמאשרים

בודקים שזה נקלט

פקודה שרצה אוטומטית לאורך קבוע. אריפו, לא לקשר.	מוסד את הסקירה, ואחסכר הציע כ-ebnalC.
ebnalC הלשיר לו שהצא שאשער ראדוק הוקציה.	כאשר שמיטה ebnalC לא לקשר הוכרה, שלא לשלחה לכי יפתח.

נועה רוצה לוודא ש-Claude לעולם לא יערוך את סיכומי ההרצאות המקוריים. מה הכי נכון?

א לכתוב ב-CLAUDE.md "בבקשה אל תערוך"

ב הוק PreToolUse שחוסם עריכה בתיקייה, עם קוד יציאה 2

ג להריץ /loop כל 5 דקות

ד לכתוב בקשה אחרי כל תשובה

כתבתם: "/goal הסיכום טוב". מה הבעיה העיקרית?

- **א** המטרה ארוכה מדי
- **ב** התנאי לא מדיד, השופט לא יודע מתי הוא התקיים
- **ג** צריך להריץ אותה פעמיים
- **ד** אי אפשר להשתמש ב- /goal בעברית

מה עכשיו יש לכם

✔ למדנו

שישה דרכים להתחיל עבודה בלי לבקש: הוק (אירוע), /loop (זמן), /goal (תנאי), claude -p (סקריפט), ערוץ (הודעה), וקישור (לחיצה).

✔ בנינו

אוטומציה אחת שרצה אצלכם: שומר על הקבצים, בדיקה לפי שעון, או מטרה שנסגרת רק כשהמשימה באמת גמורה.

אבל...

עכשיו יש לכם הרבה חלקים: הוראות, סוכנים, הוקים, תזמון. איך מחברים אותם לתהליך אחד ומשתפים אותו עם אחרים? **זה מודול 9: מחברים הכול ומשתפים ב-GitHub.**